

HTML + CSS Basics

Enough frontend to build a real Twitter UI — the backend developer's survival kit.

01

Why we're here

What Track B is for, where it sits in the course, and the honest expectation.

We're backend engineers learning **just enough** frontend.

The goal

Build and understand the UI layer of the Twitter product you are shipping.

Not the goal

Becoming a frontend specialist or memorising every CSS property.

This class is about literacy — reading the UI code, knowing what it does, and knowing when it's wrong.

Where today fits

Class 3	HTML + CSS — today	the skeleton & skin
Class 6	JavaScript + ES6+	the behaviour
Class 9	Next.js — first Twitter UI deployed to Vercel	it goes live
Class 12	Next.js — components, state, routing	it gets real
Class 22	Next.js calls the real .NET API	full stack 🎉

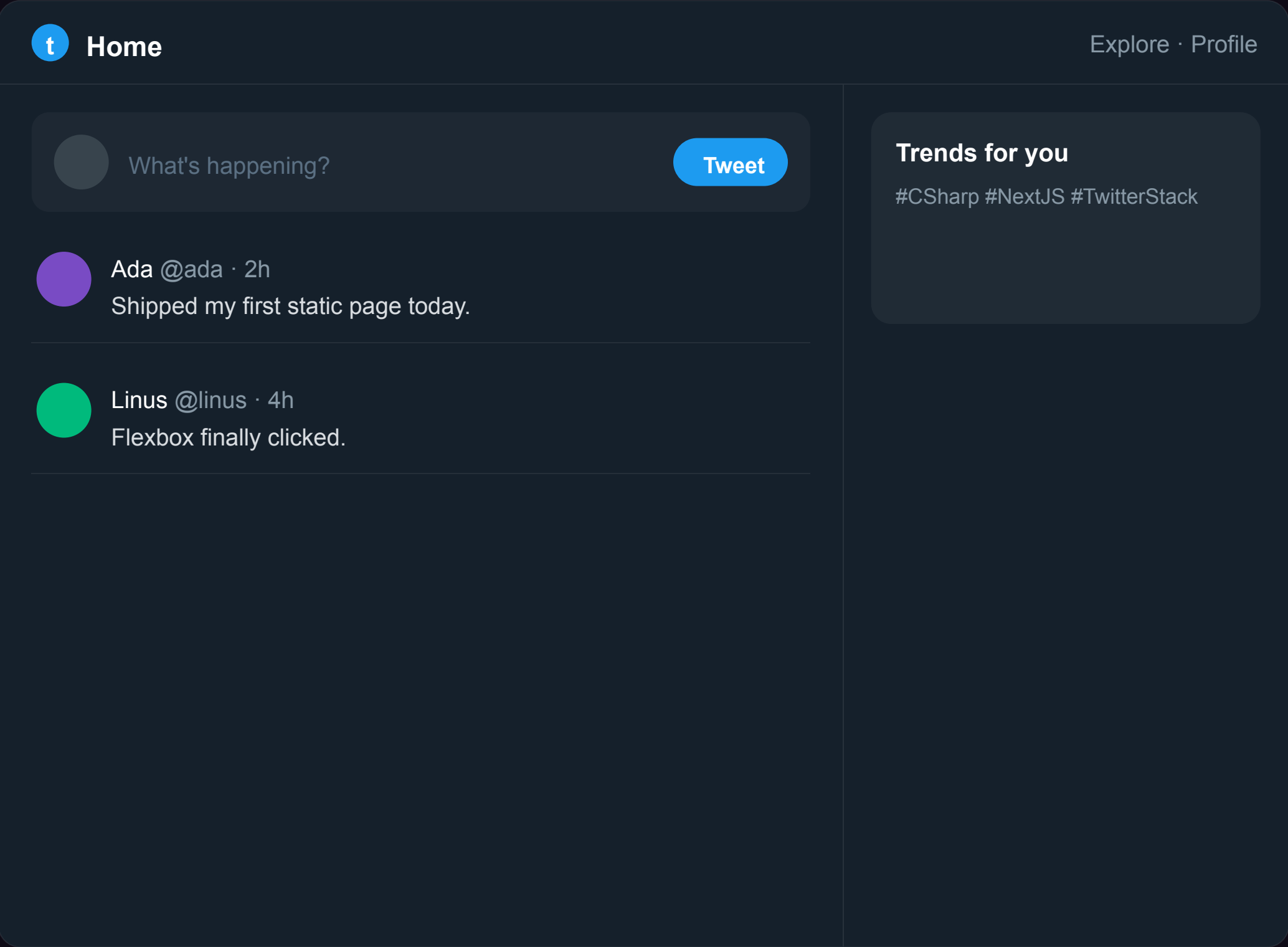
PART 1 · WHY THIS MATTERS

Soon, AI will write most of your CSS.

After Class 9 you'll describe a feature and it generates the React + Tailwind. Your job becomes [reading it, fixing it, and integrating it](#).

You can't do that without knowing HTML and CSS. Today buys you that literacy.

What you'll have built by the end



Top nav bar

Logo + nav links, laid out with Flexbox.

Compose box

A form: avatar, textarea, Tweet button.

Tweet cards ×3

Semantic articles, styled with the box model.

Trends sidebar

An aside with a list of topics.

02

How the browser works

From a text file to pixels on screen — the DOM, in one slide.

PART 2 · THE MENTAL MODEL

Your file becomes a **tree**, then pixels

1 · HTML file

```
<body>
  <h1>Hi</h1>
  <p>Feed</p>
</body>
```



2 · DOM tree (in memory)

body

```
└─ h1 → "Hi"
   p → "Feed"
```



3 · Rendered

Hi
Feed

Why care? JavaScript (Class 6) and React (Class 9) both work by changing this tree. Today we write the HTML that becomes it.

03

HTML fundamentals

Semantic structure, the tags you'll use constantly, and forms.

Semantic HTML: the right tag for the job

Use `<header>` for a header, not a generic `<div>`. Same pixels — but the meaning is now in the code.

Structure

Code reads like an outline
— anyone can follow the page.

SEO & a11y

Search engines and
screen readers
understand the page.

Maps to React

JSX components are
these same elements.
Learn once, reuse at
Class 9.

The skeleton of every page

```
<!DOCTYPE html>
<html>
  <head>           // metadata
    <title>TwitterStack</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>           // visible content
    <h1>Home</h1>
  </body>
</html>
```

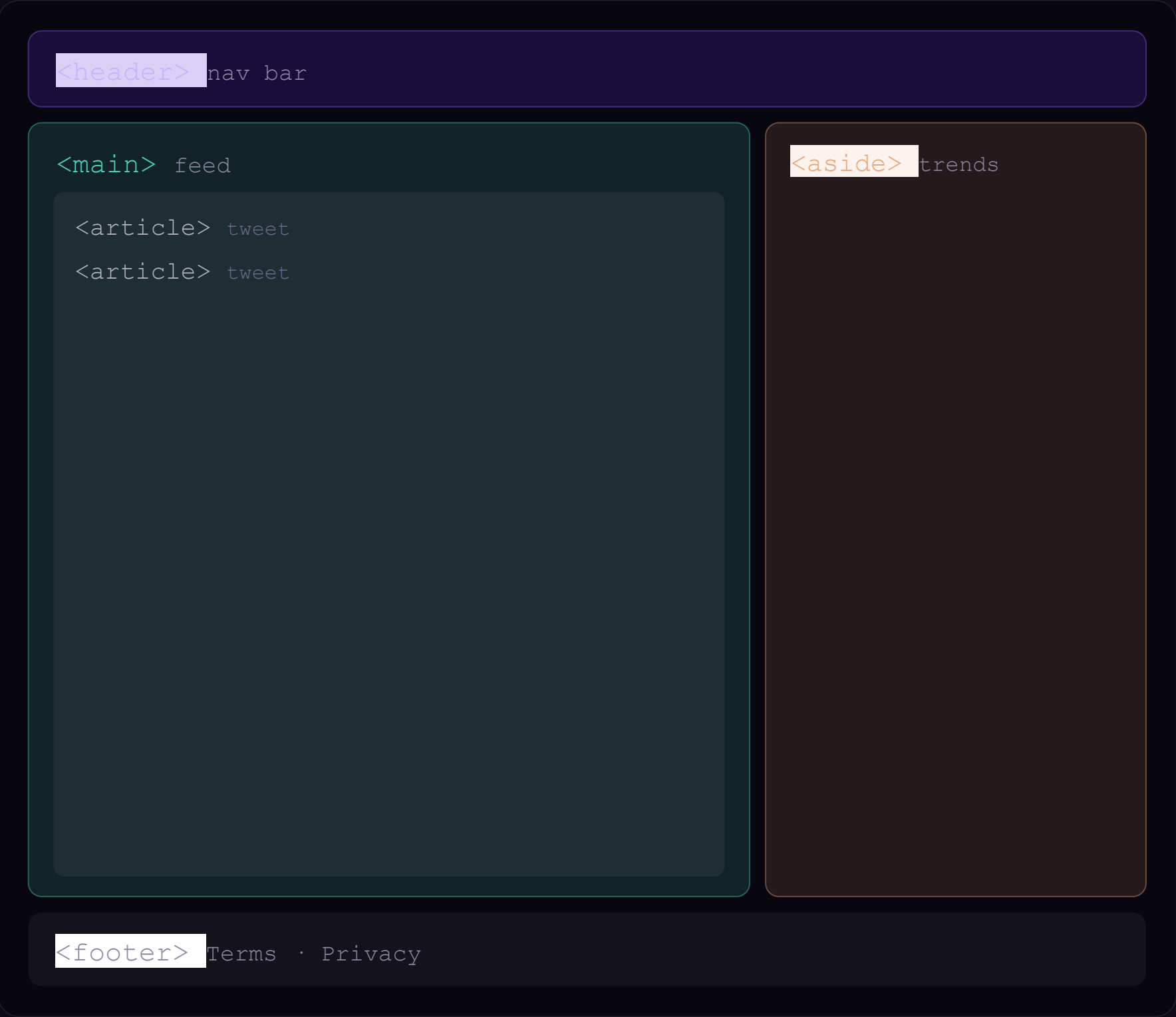
<head>

Invisible metadata — title, charset, and links to your CSS.

<body>

Everything the user actually sees on the page.

Semantic tags, mapped to Twitter



- `<header>` top nav bar — logo + links
- `<nav>` the links inside the header
- `<main>` the tweet feed area
- `<section>` a logical grouping
- `<article>` one self-contained tweet
- `<aside>` the trends sidebar
- `<footer>` bottom links

Content tags you'll use constantly

`<h1>–<h6>` headings — only one h1 per page

`<p>` paragraph — tweet content

`<a href>` links — @handles, #hashtags

`<img alt>` images — avatar, media

`<ul><ol><li>`
`>` lists — nav items, trends

`<span>` inline wrapper — style part of text

`<div>` generic block container — last resort, **not semantic**. Reach for a real tag first.

Block vs. inline

Block

Takes the full width, starts on a new line. `div` · `p` · `h1` · `article`



Inline

Flows with the text, only as wide as its content. `span` · `a` · `img`



The feed skeleton — structure only

```
<header>
  <nav> TwitterStack · Home · Explore </nav>
</header>
<main>
  <section>
    <article> <p>First tweet</p> </article>
    <article> <p>Second tweet</p> </article>
  </section>
</main>
<aside> <h2>Trends</h2> </aside>
<footer> Terms · Privacy </footer>
```

PART 3 · SECTION B

Forms

How users give input. On Twitter that's the [login page](#), the [compose box](#), and the [search bar](#) .

We'll build the compose box.

The compose box, element by element

```
<form>
  <label for="tweet">New Tweet</label>
  <textarea id="tweet"
    placeholder="What's happening?"
    required></textarea>
  <button type="submit">Tweet</button>
</form>
```

`<form>` — the container

`<label for>` — pair with an input

`<input type>` — text · email · password

`<textarea>` — multi-line, the tweet box

`<button submit>` — posts the tweet

Attributes that matter

name & id

id links the label and is JavaScript's handle. name is the key sent to the server.

**placeholder vs
value**

placeholder is grey hint text that vanishes on typing. value is real pre-filled content.

required

Basic HTML-level validation — the form won't submit while the field is empty. Free, no JavaScript.

Structure is done. Next: CSS — making it actually look like Twitter.

HALFWAY

**5-minute
break**

04

CSS fundamentals

The cascade, the box model, Flexbox, and just enough typography.

One rule, three ways to attach it

```
.tweet {           /* selector */
  color: #0F1419;   /* declaration */
  padding: 16px;
}
```

Inline

`style="..."` — avoid, unmaintainable.

`<style>` tag

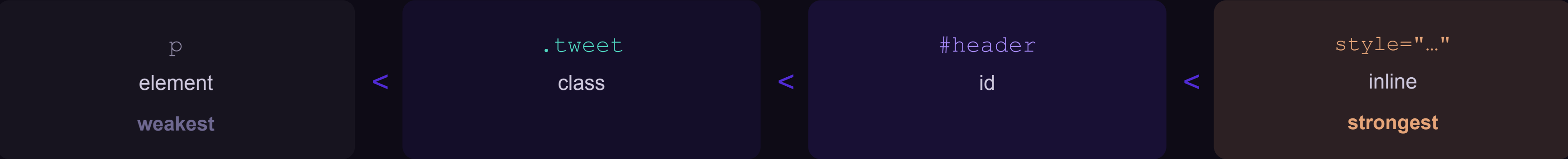
In the head — fine for tiny demos.

External file

`style.css` — the right way. What we use.

When rules conflict: **specificity** wins

Two rules target the same element? The more specific one applies. On a tie, the later rule wins — that's the cascade.



Selectors & inheritance

<code>p</code>	every <code><p></code> element
<code>.tweet</code>	<code>class="tweet"</code>
<code>#header</code>	<code>id="header"</code>
<code>.tweet p</code>	<code><p></code> inside <code>.tweet</code>
<code>a:hover</code>	<code><a></code> on mouse-over

Inherits ✓

font-family, font-size, color — set once on `body`.

Doesn't ✗

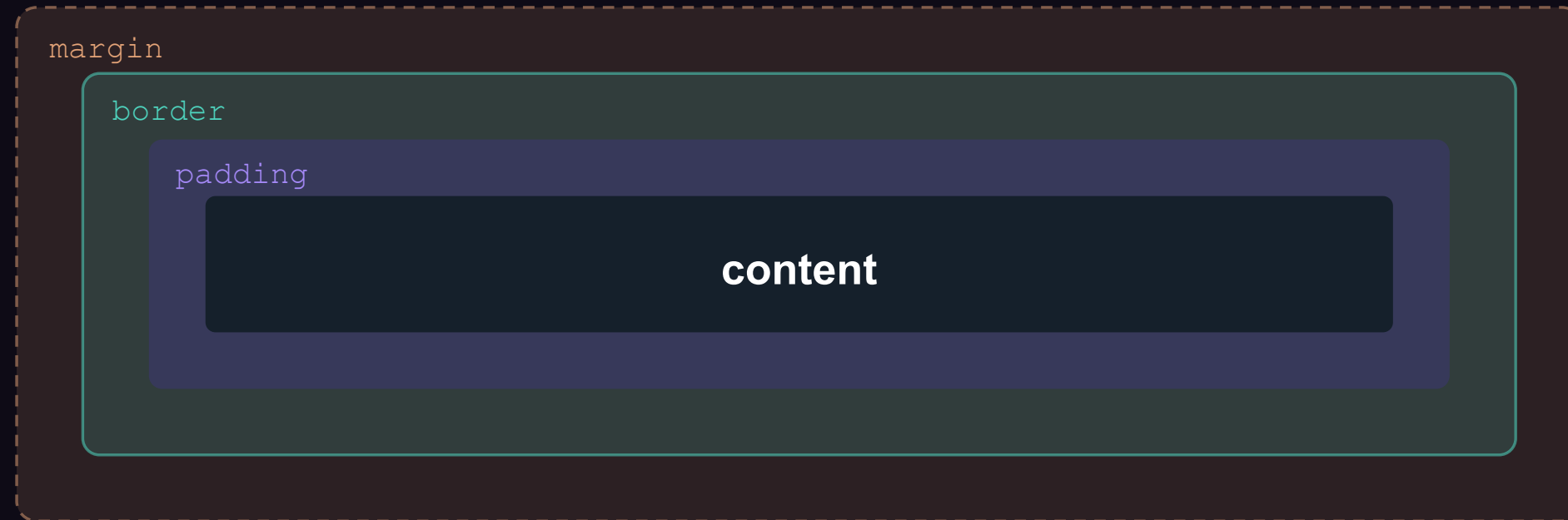
margin, padding, border — set per element.

PART 4 · SECTION B · THE NON-NEGOTIABLE

If you remember one thing from CSS — it's the **box model**.

Every element on the page is a rectangular box. Understand its four layers and layout stops being mysterious.

Four layers, inside out



content— the text or image itself

padding — space inside, content → border

border— the visible edge

margin — space outside, between boxes

The classic mix-up: padding pushes the border out; margin pushes neighbours away.

Always use **border-box**

```
/* top of every stylesheet */  
* { box-sizing: border-box; }
```

Default (content-box)

width = content only. Padding + border make the box bigger than you set. Surprising.

border-box

width includes padding + border. What you set is what you get. Intuitive.

Flexbox

Turn a parent into a [flex container](#) , then align its children along two axes. Rows and columns become trivial.

Six properties do the work

<code>display: flex</code>	makes the parent a flex container
----------------------------	-----------------------------------

<code>flex-direction</code>	row (horizontal) or column (vertical)
-----------------------------	---------------------------------------

<code>justify-content</code>	main axis — start · center · space-between
------------------------------	--

<code>align-items</code>	cross axis — center is the workhorse
--------------------------	--------------------------------------

<code>gap</code>	space between children — no more margins
------------------	--

<code>flex-wrap</code>	let children wrap to the next line
------------------------	------------------------------------

Three Flexbox patterns build the page

Nav bar

display: flex · justify-content: space-between

Tweet card — avatar left, content right

display: flex + gap

display: flex · gap: 12px

Reply

Retweet

Like

Share

display: flex · gap · align-items: center

Typography — the properties you'll set

```
body {  
  font-family: system-ui, sans-serif;  
  font-size: 15px;  
  line-height: 1.4;  
  color: #0F1419;  
}  
a { text-decoration: none; }
```

`font-family` — the typeface

`font-size` / `weight` — scale & boldness

`line-height` — readability

`color` — text colour

`text-decoration: none` — kill underlines

Colour & sizing units

Colour



#15202B hex — Twitter dark



rgb(29,155,240) rgb



white named

Units

px — a fixed pixel

rem — relative to root font size · default to this

% — relative to the parent

Responsive — just the idea

```
max-width: 600px
```

cap the feed width on wide screens

```
width: 100%
```

fill the space on smaller ones

```
/* apply rules only below 768px */
@media (max-width: 768px) {
  aside { display: none; } /* hide sidebar */
}
```

We'll revisit all of this with [Tailwind](#) in Next.js — no memorising today.

05

Live build

Put it all together — a static Twitter feed page, from scratch. Type along.

Assemble the page

1 Top nav bar + two-column layout

header with logo & links; main feed + aside sidebar, laid out with Flexbox.

2 Compose box

avatar + textarea + Tweet button, as a form.

3 Three tweet cards + trends sidebar

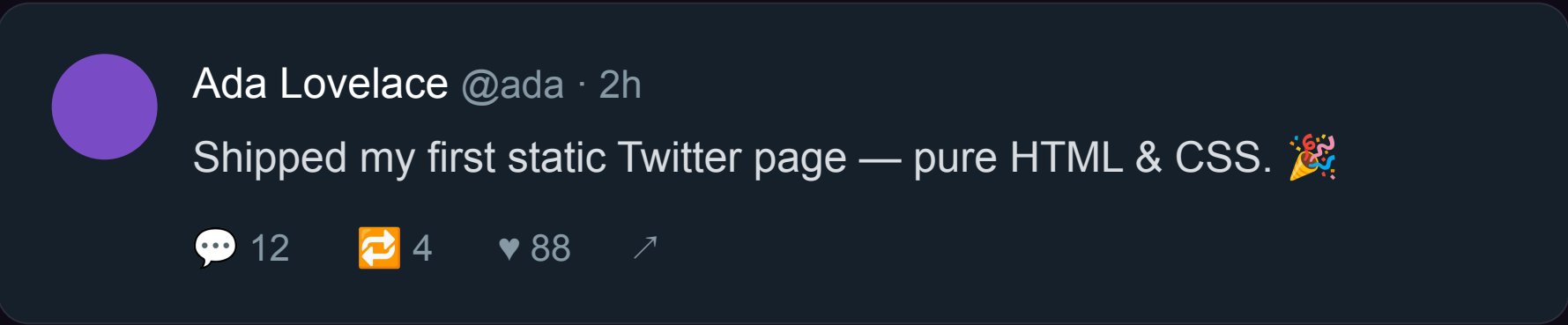
each card: avatar, name, @handle, text, timestamp, action row.



Theme with CSS variables

`var(--bg)` — change one line, re-theme everything.

One tweet card = everything today



`<article>`— semantic, self-contained

`display: flex; gap`— avatar | content

`border-radius: 50%`— the circular avatar

`padding`— the box model, breathing room

`action row: flex + gap`— icons in a line

PART 5 · SHIP IT

Push it to GitHub — you built a real page

Same discipline as every class: commit and push. Keep this page — it's the direct ancestor of the Next.js Twitter UI you rebuild at Class 9.

```
git add . · git commit -m "Static Twitter feed" · git push
```

06

Wrap-up

The two things to remember, your homework, and what's next.

The two things to remember

01

The box model

Content, padding, border, margin. Every element is a box.

02

Flexbox

display: flex, then align children on two axes.

The honest reminder: AI will generate most of your CSS in this course. Your job is to read it and know when it's wrong.

Extend the page you built

1 Finish the feed

`flexbox` · `box model`

Complete all three tweet cards with avatar, name, @handle, text, timestamp, and an action row.

2 Build a login page

`forms` · `labels`

A second page: a form with labelled `email` & `password` inputs and a `required` submit.



Stretch — re-theme to light mode

Switch the page to a light theme by changing only your CSS variables. Proves you understood `var()` & the cascade.

Submit: push to a `class-03/` folder → branch `class-03` → PR titled `[Class 03] Your Name — HTML & CSS`

OPEN FLOOR

Questions?

Anything from today — semantic tags, the box model, Flexbox, forms. The homework is where it becomes muscle memory.

WHAT'S NEXT

Class 6 — JavaScript brings this page to life

Track A continues at Class 4 with the .NET ecosystem & CLR. Track B returns at Class 6 to add behaviour to the very page you built today.

SEE YOU NEXT CLASS

TWITTERSTACK · TRACK B